

Relatório de Atuação dos Integrantes

Algoritmo de Similaridade de Programas inspirado em Bag-of-Words

Adryel Rodrigues

Amon Sinder

Lucas Silva

1 Identificação

Item	Descrição
Trabalho	Algoritmo de similaridade de programas inspirado em Bag-of-Words.
Linguagem	Haskell.
Objetivo	Comparar dois arquivos de código-fonte por frequência ponderada de palavras.
Integrantes	Adryel Rodrigues, Amon Sinder e Lucas Silva.

2 Resumo da solução

O programa lê quatro arquivos de entrada:

- `res.txt`: palavras reservadas da linguagem;
- `sep.txt`: caracteres separadores;
- `c1.txt`: primeiro código-fonte;
- `c2.txt`: segundo código-fonte.

A solução remove os separadores, divide os códigos em palavras, calcula frequências ponderadas e compara as frequências de `c1` com as de `c2`. Palavras reservadas têm peso 2; as demais têm peso 1.

O índice final calculado é:

$$\text{similaridade} = \frac{m}{\sum f1}$$

em que $\sum f1$ é a soma das frequências ponderadas de `c1`, e m soma os valores de `f1` cuja diferença em relação a `f2` é de até 10% de `f1`.

3 Arquivos entregues

Arquivo	Função
Main.hs	Código-fonte principal em Haskell.
Makefile	Comandos de compilação, execução e limpeza.
res.txt	Palavras reservadas da linguagem usada na comparação.
sep.txt	Caracteres separadores descartados na tokenização.
c1.txt	Primeiro código-fonte de exemplo.
c2.txt	Segundo código-fonte de exemplo.
saida_esperada.txt	Saída esperada para os exemplos.
README.md	Instruções de execução e descrição do projeto.
materiais_auxiliares/APRESENTACAO.md	Apresentação e preparação da apresentação.
materiais_auxiliares/APRENDIZADO_HASKELL.md	Material de estudo sobre Haskell e sobre o código.
materiais_auxiliares/RELATÓRIO_INTEGRANTES.tex	Relatório de integrantes.

4 Funcionamento implementado

4.1 Tokenização

Os caracteres presentes em `sep.txt` são substituídos por espaço antes da separação em palavras. Isso evita que expressões como `x=10` ou `x==y` sejam lidas como uma única palavra.

<code>x=10</code>	<code>-> x 10</code>
<code>x==y</code>	<code>-> x y</code>
<code>foo(bar)</code>	<code>-> foo bar</code>

4.2 Frequência ponderada

Caso	Peso aplicado
Palavra em <code>res.txt</code>	2 por ocorrência.
Demais palavras	1 por ocorrência.

4.3 Regra dos 10%

Para cada palavra de `c1`, o programa obtém:

- `f1`: frequência ponderada em `c1`;
- `f2`: frequência ponderada da mesma palavra em `c2`.

A palavra contribui para m quando:

$$|f1 - f2| \leq 0,1 \times f1$$

No código, essa regra é implementada como:

```
abs (freq1 - freq2) * 10 <= freq1
```

5 Exemplo de saída

Para os arquivos de exemplo atuais, a saída esperada é:

```
--- Relatrio de Frequncias (c1) ---
let: 4
x: 3
y: 3
do: 2
else: 2
if: 2
return: 2
then: 2
10: 1
20: 1
main: 1
-----
m = 17
Soma(f1) = 23
ndice de Similaridade = 0.7391304347826086
```

6 Atuação dos integrantes

O desenvolvimento foi colaborativo. Todos os integrantes participaram da compreensão do enunciado, da revisão da solução, dos testes manuais e da preparação para apresentação. A divisão abaixo registra apenas uma separação geral de foco, não uma separação exclusiva de responsabilidades.

Os nomes estão em ordem alfabética.

Integrante	Foco principal registrado
Adryel Rodrigues	Documentação, organização do material de apresentação e revisão geral da explicação do trabalho.
Amon Sinder	Implementação principal em Haskell e consolidação das regras do algoritmo no código.
Lucas Silva	Organização dos arquivos de exemplo, validação da saída esperada e roteiro de compilação/execução.

7 Conformidade com o enunciado

Exigência	Situação
Código-fonte em Haskell	Atendido em <code>Main.hs</code> .
Arquivo de palavras reservadas	Atendido em <code>res.txt</code> .
Arquivo de separadores	Atendido em <code>sep.txt</code> .
Dois arquivos de código a comparar	Atendido em <code>c1.txt</code> e <code>c2.txt</code> .
Cálculo de frequências	Atendido em <code>getFrequencies</code> .
Peso dobrado para reservadas	Atendido em <code>wordWeight</code> .
Frequências de <code>c1</code> em ordem decrescente	Atendido com <code>sortBy</code> e <code>frequencyOrder</code> .
Desempate lexicográfico	Atendido em <code>frequencyOrder</code> .
Cálculo de <code>m</code>	Atendido em <code>matchingScore</code> .
Índice <code>m / soma(f1)</code>	Atendido em <code>similarityIndex</code> .
Makefile ou roteiro de execução	Atendido em <code>Makefile</code> e <code>README.md</code> .
Arquivos de exemplo	Atendido em <code>res.txt</code> , <code>sep.txt</code> , <code>c1.txt</code> , <code>c2.txt</code> e <code>saida_esperada.txt</code> .
Apresentação	Atendido em <code>materiais_auxiliares/APRESENTACAO.md</code> .
Relatório de atuação	Atendido neste arquivo.

8 Como executar

Compilar:

```
make build
```

Executar com os exemplos:

```
make run
```

Execução manual equivalente:

```
./similaridade res.txt sep.txt c1.txt c2.txt
```

Limpar arquivos gerados:

```
make clean
```

9 Limitações

O algoritmo mede similaridade lexical por frequência de palavras. Ele não faz análise sintática completa nem análise semântica. Portanto:

- não garante detecção de plágio;
- não identifica equivalência semântica entre programas;
- pode ser afetado por renomeação de variáveis;
- não trata comentários e strings de forma especial.

10 Conclusão

O projeto atende aos requisitos do enunciado: lê os quatro arquivos pedidos, calcula frequências ponderadas, imprime as frequências de `c1`, calcula o valor `m` e informa o índice de similaridade. A implementação foi mantida pequena para facilitar revisão, apresentação e entendimento por todos os integrantes.